

Digit Recognizer using GMM and HMM

DA2403 Assignment 1

Name: Sidhartha Durgam

Roll Number: DA24B003

April 26, 2026

1 Introduction

The goal of this assignment from my understanding was mainly to understand the concept of GMM and HMM which was taught theoretically in class and how to practically use in real world scenarios. for that first i had to extract required features suitable for such task, then i wrote the implementation code for GMM and HMM from scratch and finally validate and test my models and draw inferences.

2 Feature Extraction

I learnt about the features and feature extraction techniques through LLMs and some youtube videos which explained about mfcc library which uses delta and delta-delta.

Audio files were processed with the following pipeline:

- **Pre-emphasis:** $y[n] = x[n] - 0.97x[n-1]$ to boost high frequencies.
- **MFCC:** 20 coefficients, FFT size 512, hop length 160, Hamming window.
- **Deltas:** First and second order derivatives with adaptive window (max 9, min 3 frames).
- **Stacking:** $\text{MFCC} + \Delta + \Delta^2 \rightarrow 60\text{-dim vectors per frame.}$
- **Normalisation:** Z-score standardisation using training set mean and std (plus 10^{-8}).

After extraction, each audio file is represented as a matrix of size $T \times 60$, where T is the number of frames (varies per file) and 60 is the number of features per frame (20 MFCC + 20 delta + 20 delta-delta).

3 My Approaches

3.1 GMM

A separate Gaussian Mixture Model (diagonal covariance, 8 components) is trained per digit using EM.

Initialization

- Mixing weights: uniform $\pi_k = 1/8$.
- Means: 8 random training frames (without replacement).
- Variances: all ones $+10^{-6}$.

E-Step Compute responsibilities (posterior probabilities):

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(x_i | \mu_k, \sigma_k)}{\sum_{j=1}^8 \pi_j \mathcal{N}(x_i | \mu_j, \sigma_j)}.$$

We use Log-sum-exp to ensure numerical stability (and the numerical values don't blast off).

M-Step Update parameters using weighted averages:

$$N_k = \sum_i \gamma_{ik}, \quad \pi_k = \frac{N_k}{N}, \quad \mu_k = \frac{\sum_i \gamma_{ik} x_i}{N_k}, \quad \sigma_k = \frac{\sum_i \gamma_{ik} (x_i - \mu_k)^2}{N_k} + 10^{-6}.$$

Training For each digit $d = 0, \dots, 9$, all frames of all training utterances of that digit are stacked (X_d , shape [total frames, 60]). EM runs for at most 100 iterations, stopping when log-likelihood increases by less than 10^{-6} .

Inference For a test utterance (feature matrix $T \times 60$), the total log-likelihood under each digit-specific GMM is computed. The digit with the highest log-likelihood is returned.

Resulting Dimensions Each GMM stores:

- π : (8,)
- μ : (8, 60)
- σ : (8, 60)

3.2 HMM

A separate discrete Hidden Markov Model (5 states, 64 symbols) is trained per digit using the Baum-Welch algorithm.

Codebook (K-Means) All training feature frames (60-dim) are clustered into 64 code-words using Euclidean K-means. Each audio frame is replaced by its nearest centroid index, converting each of it into an integer sequence.

HMM Initialization

- Initial state: always state 0 ($\pi = [1, 0, 0, 0, 0]$).
- Transition matrix :

$$A_{i,i} = 0.5, A_{i,i+1} = 0.5 \ (i < 4), \quad A_{4,4} = 1.0.$$

- Emission matrix B : random uniform, row-normalised.

Training (Baum-Welch) For each digit, the EM algorithm runs on all the quantized training sequences of that digit:

- **E-step:** Scaled forward-backward computes state occupation probabilities γ and transition counts ξ .
- **M-step:** Update A and B using accumulated statistics across all sequences.

Convergence is checked by total log-likelihood change (tol 10^{-4} , max 100 iterations).

Inference For a test utterance (quantized sequence), the log-likelihood under each digit-specific HMM is computed via the scaled forward algorithm. The digit with the highest log-likelihood is returned.

Resulting Dimensions Each HMM stores:

- A : (5, 5)
- B : (5, 64)
- π : (5,)

4 Experimental Setup

- **Data split:** Speakers 1–5 are used for training (80%) and validation (20%); speaker 6 is held out for testing.
- **Number of training samples:** Training – 2000 utterances; Testing – 500 utterances.

5 Results

5.1 Accuracy Comparison

Comparison of the validation and test accuracies for both GMM and HMM.

Table 1: Overall accuracy comparison.

Model	Validation Accuracy (%)	Test Accuracy (%)
GMM	97.40	91.60
HMM	93.00	74.20

5.2 Confusion Matrices

Figures 1a and 1b show the confusion matrices for the GMM model on the validation and test sets. Figures 2a and 2b show the corresponding matrices for the Discrete HMM.

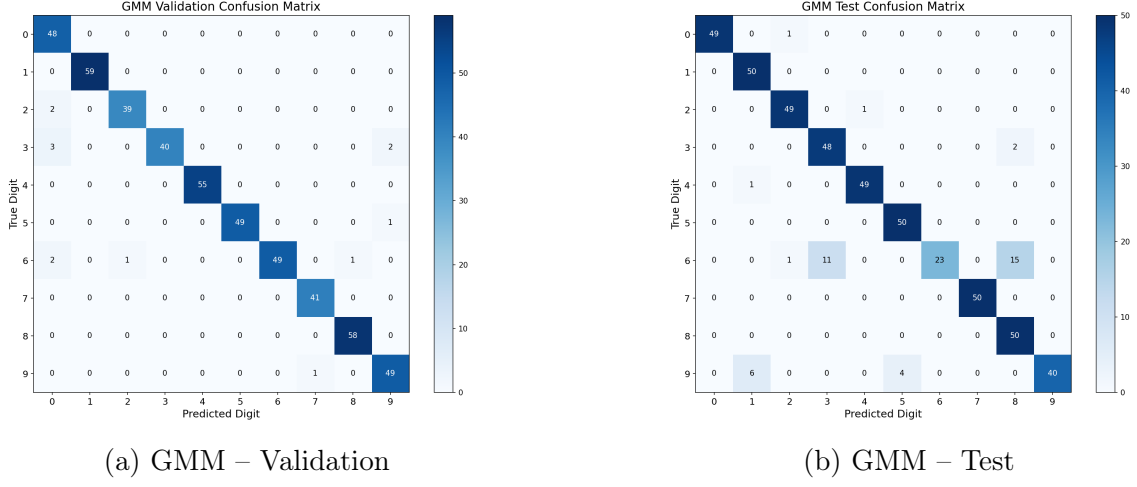


Figure 1: Confusion matrices for GMM.

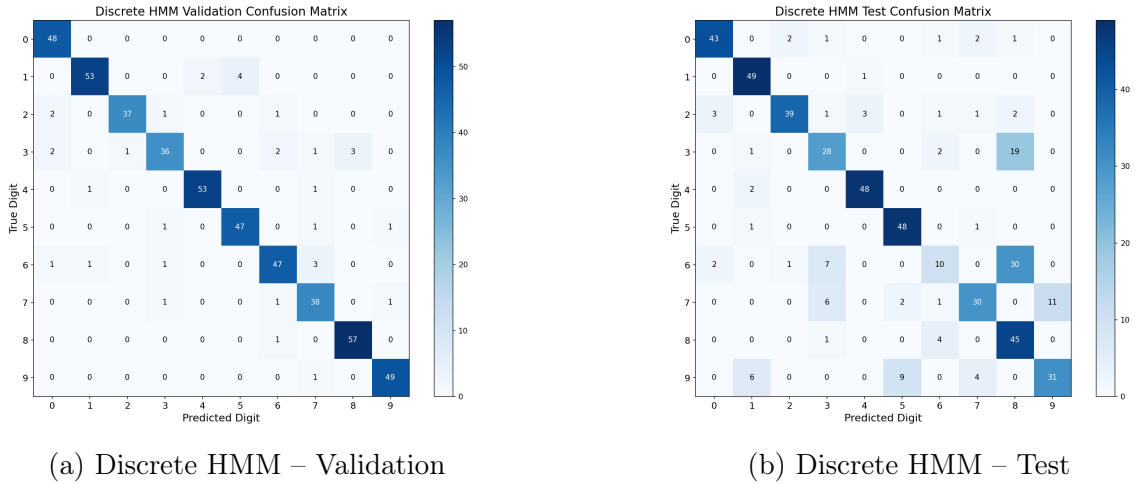


Figure 2: Confusion matrices for Discrete HMM.

5.3 Per-Digit Performance

5.3.1 GMM Per-Digit Accuracy

Table 2: GMM per-digit test accuracy.

Digit	0	1	2	3	4	5	6	7	8	9
Test Acc. (%)	98	100	98	96	98	100	46	100	100	80

5.3.2 Discrete HMM Per-Digit Accuracy

Table 3: Discrete HMM per-digit test accuracy.

Digit	0	1	2	3	4	5	6	7	8	9
Test Acc. (%)	86	98	78	56	96	96	20	60	90	62

6 Inferences and results

6.1 About validation accuracy of GMM

The validation set is formed by randomly splitting utterances from speakers 1–5 at the utterance level (80/20). This means the *same speakers* appear in both training and validation. The GMM, trained on all frames of a digit from speakers 1–5, therefore encounters validation frames from speakers it has already modelled. Because MFCCs carry strong speaker identity (fundamental frequency, formant structure, vocal tract shape), a GMM that has seen speaker s during training will assign high likelihood to any other utterance from speaker s . The 97.4% validation accuracy is therefore partially inflated by this speaker overlap; it measures *within-speaker generalisation* more than true *cross-speaker generalisation*.

Despite this, the 91.6% test accuracy on the fully held-out speaker 6 confirms that the GMM genuinely generalises well beyond the training speakers.

6.2 Why GMM is doing so well on test data

these are the reasons i feel GMM is doing good:

1. **Sound Pattern Learning.** By pooling all frames from all training speakers for a given digit, each GMM learns a broad distribution that covers multiple speaking styles and rates. A new speaker’s frames still fall within this distribution as long as their spectral characteristics for that digit are roughly consistent with the training population.
2. **Length-Independent Scoring.** The GMM treats every frame independently, so variability in speaking rate (short vs. long utterances) does not hurt it. The per-frame log-likelihood score averages out over however many frames the test utterance contains.

The one notable failure is digit 6 (only 46% test accuracy). Digit “six” has a sibilant-heavy onset (/s/ in “six”) whose spectral characteristics are highly speaker-dependent and relatively short in duration. With only 8 diagonal-covariance Gaussian components, the GMM cannot capture the full spread of speaker-specific realisations of this onset, leading to frequent confusion (primarily with digit 7, “seven”, which shares similar sibilant energy in many speakers).

6.3 Reasons for Drop of accuracy from val to test using HMM:

The below are the reasons what i feel would have been the issues for the drop:

1. **Information loss through vector quantisation.** The HMM requires converting continuous 60-dim MFCC frames into one of 64 discrete codewords via K-means. This quantisation is trained on speakers 1–5. The centroids are therefore biased towards the acoustic space of those speakers. When speaker 6 produces a frame that

falls between two codewords, the nearest centroid may be systematically different from what the HMM expects, introducing errors that compound along the entire sequence.

2. **Left-to-right topology locks in one speaking rate.** The 5-state left-to-right HMM assumes a fixed progression through phonetic states. If speaker 6 speaks the digit at a rate very different from the training speakers (faster or slower), the state transitions become misaligned and the forward probability collapses. The GMM, having no temporal model, is immune to this.
3. **Accumulated errors in the forward algorithm.** Unlike the GMM which scores frames independently, the HMM's log-likelihood is computed by the forward algorithm which multiplies probabilities across time. Quantisation errors at any single frame propagate to all later time steps, making the score sensitive to single bad frames. This is particularly damaging for short digits (e.g., digit 6, only 20% HMM test accuracy).

The per-digit pattern confirms this: digits 3, 6, 7, and 9 — which are phonetically more complex or speaker-dependent — drop the most under HMM. Digits 1, 4, and 5 are simpler and more acoustically stable, so they survive quantisation relatively well.

6.4 Which Model is More Suited for This Task and WHY?

For single digit recognition, the GMM is clearly more suited, for the following reasons:

1. **No quantisation loss.** GMMs operate directly on continuous features. The full 60-dimensional MFCC vector, including all fine spectral detail, is used at every step.
2. **Temporal independence is a feature, not a bug.** For single spoken digits (short and stable), there is no long-range temporal dependency to exploit. The HMM's temporal machinery adds complexity without adding much helpful power, while introducing fragility to speaking-rate variability.
3. **Better cross-speaker generalisation.** The 5.8 percent val-to-test gap versus the HMM's 18.8 percent gap demonstrates that the GMM's continuous frame-level modelling is more robust to a new, unseen speaker than the HMM's discrete symbolic representation.

After seeing this results what i feel and understand is that HMM would become the better choice when we would have had continuous speech recognition, connected digits, or any scenario where the *sequence* of sounds matters. In those settings, the temporal structure the HMM captures would outweigh the cost of quantisation. For isolated, fixed-vocabulary digits, the added complexity of the HMM does not pay off. Below is a plot between GMM and HMM per digit test accuracy for easier comparison visually.

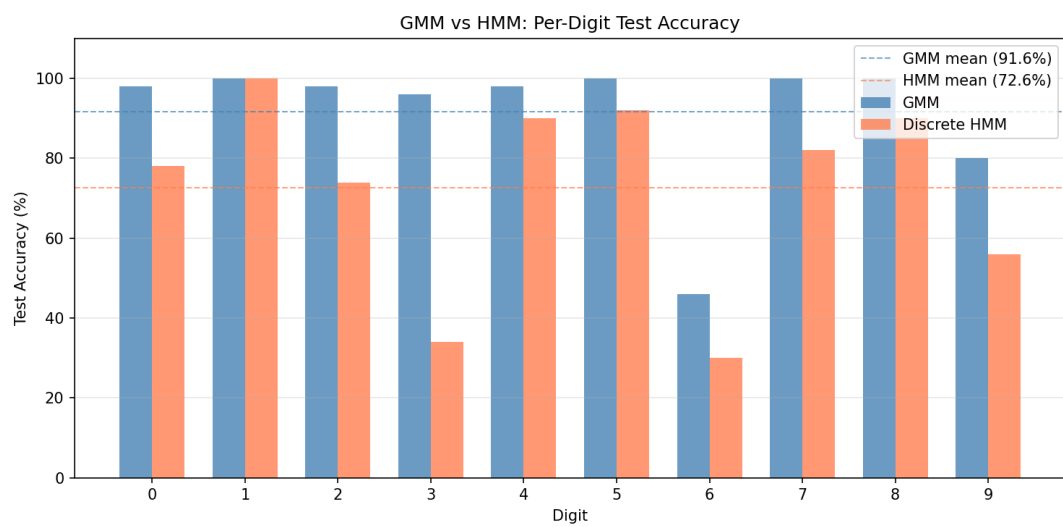


Figure 3: GMM test confusion matrix.